



## A Web tool for Automated Unit Test Generation, Test Smell Detection & Refactoring for Python

**Presented by**

**Amit Kumar Roy**

BSSE Roll: **1314**

IIT, DU

**Supervised by**

**Toukir Ahammed**

Lecturer

IIT, DU

# Why PyTestGenie ?

# Motivation

## Unit testing is essential

for software reliability and maintainability

## Manual unit test creation

time-consuming, and verifying correctness is challenging

## Gap in rule based analysis

hard to adapt to new smells or refactor

## Existing tools are limited

Often tied to specific IDEs & few support **automated refactoring**

## Agentic LLM workflows

Few tools support **multiple AI agents** working together (detector, refactorer, validator)

## Complete pipeline

for **test generation** , **test smell detection**, **automated refactoring**



What is PyTestGenie ?



## Target users:

- Developers
- Researchers
- Students/ Teachers



# Workflow...

# Normal Requirements

- Allow users to **upload Python files/folders** or **paste Python code**
- **Parse and analyze** the input source code
- **Generate unit tests** using **rule-based techniques**
- **Detect common test smells** in generated or existing test cases
- **Refactor detected test smells** to produce clean, executable unit tests

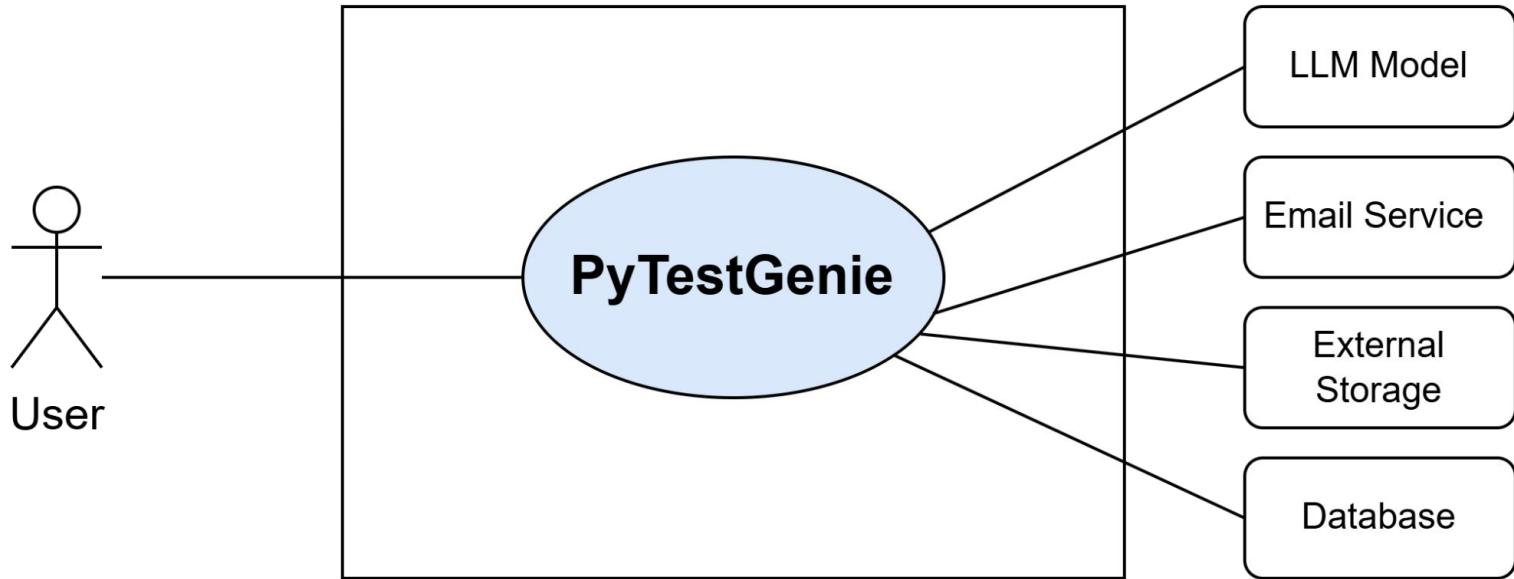
# Expected Requirements

- Support LLM-based unit test generation alongside rule-based methods
- Enable ruled-based and LLM-based approach for test smell detection and refactoring
- Provide clear explanations of detected test smells and applied refactorings
- Display analysis and refactoring results to the user
- Generate evaluation reports, including:
  - Test smell detection accuracy
  - Refactoring success rate

# Exciting Requirement

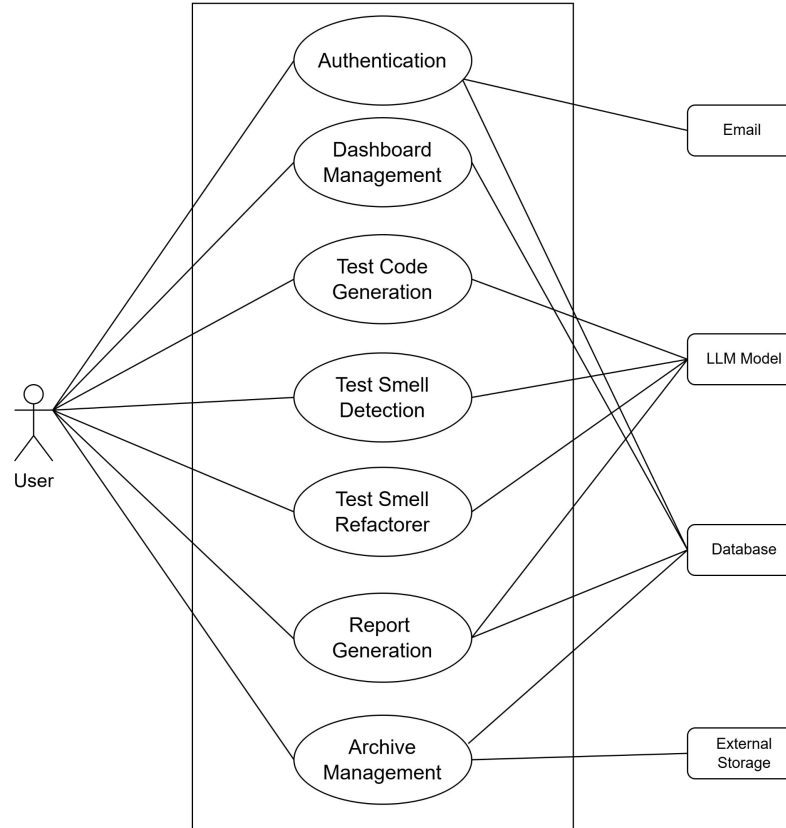
- Allow users to choose between:
  - Single-agent
  - Multi-agent (role-based, persona-driven) configurations
- Support side-by-side comparison of multiple LLMs and workflow setups
- Provide interactive user feedback to guide or refine refactoring decisions
- Extend test smell detection to multiple languages (e.g., Java, C++)
- Execute full test suites within a single integrated pipeline

## Use case diagram (Level-0)



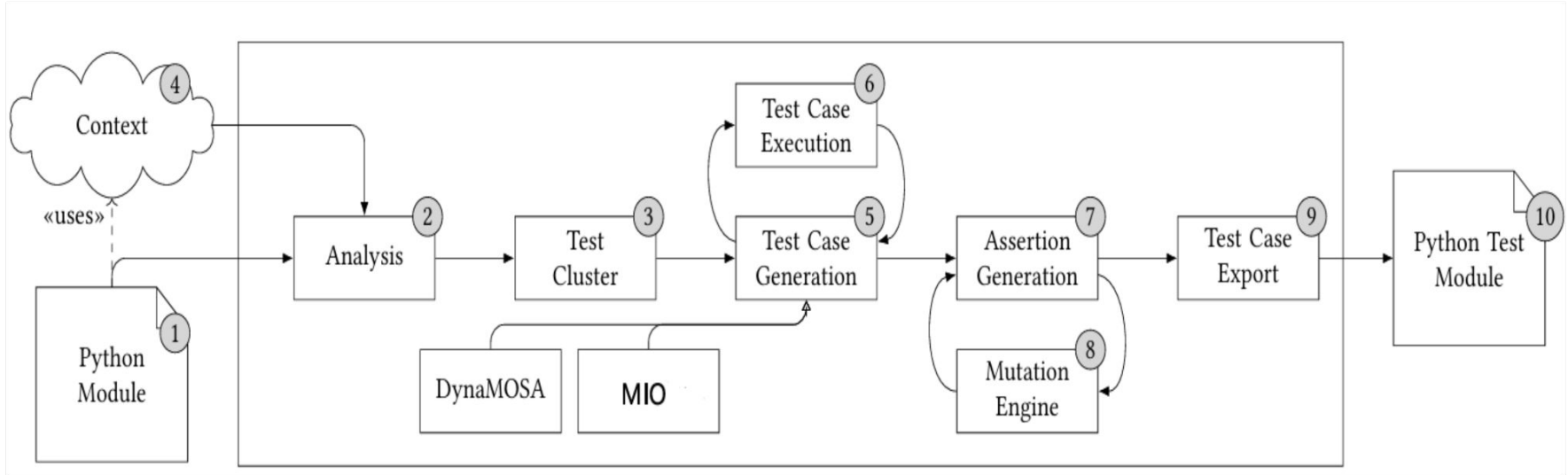
Level- 0 Use case diagram

# Use case diagram (Level-1)



Level- 1 Use case diagram

# Unit test generation



## Rule Based Approach

(Algorithm: Dynamic Many-Objective Sorting , Many Independent Objective)

## DynaMOSA Algorithm

1. Each branch or statement in the code is a goal.
2. Create random test cases.
3. Run tests and measure which goals are covered.
4. Mutate and combine tests to cover uncovered branches.
5. Ignore branches already covered; focus on remaining ones.
6. Continue until all branches are covered or time runs out.

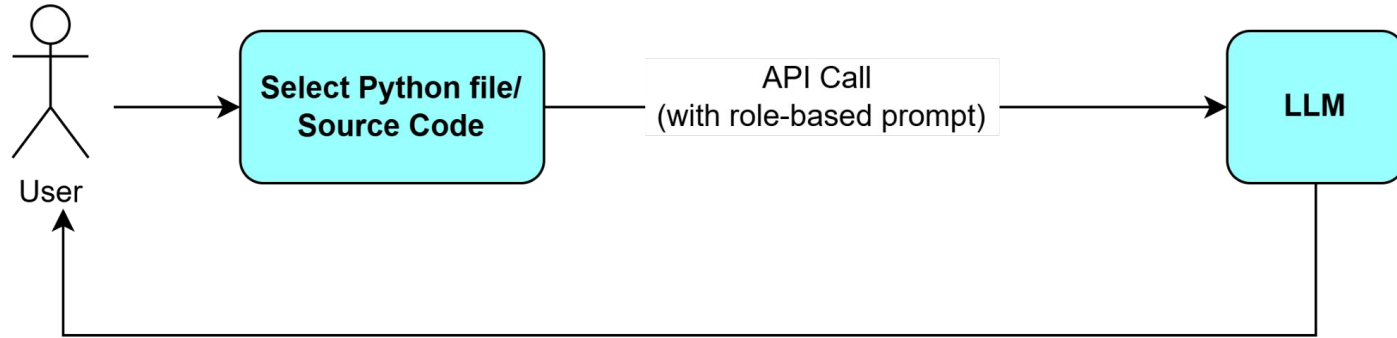
**Example:** If function has 3 branches (A, B, C).DynaMOSA first focuses on **uncovered branch A**, then B, then C, avoiding repetition.

## MIO Algorithm

1. Treat each branch as a separate objective.
2. Each goal has its own small test group.
3. Run tests for each goal and measure coverage.
4. Improve tests in each group for its goal.
5. Keep the best tests in archive for each branch.
6. Stop when all goals are covered or time ends.

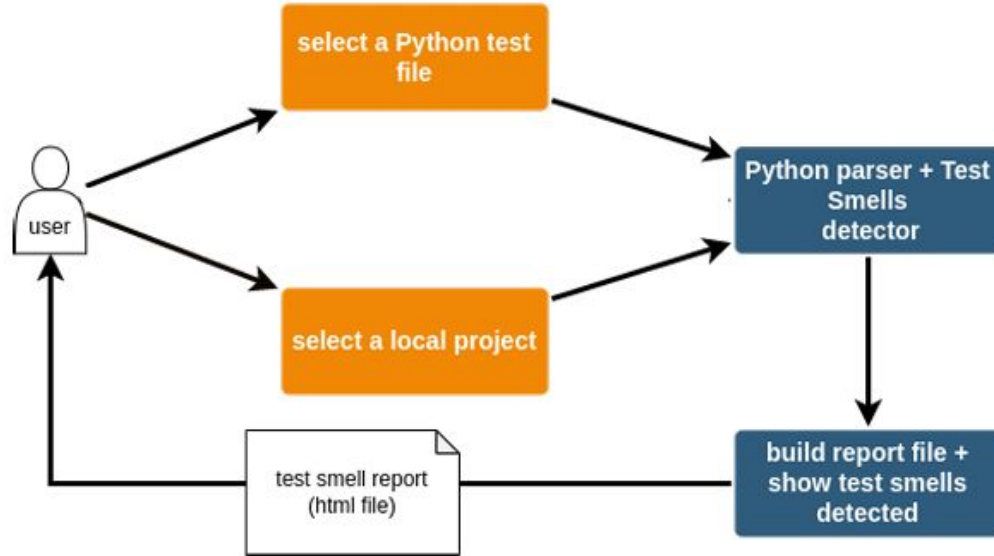
**Example:** If function has 5 branches (A, B, C, D, E), MIO maintains 5 groups of tests where each evolves to cover its branch independently, ensuring **all branches are covered efficiently**.

# Unit test generation (cont.)



LLM-Based Approach

# Test smell detection

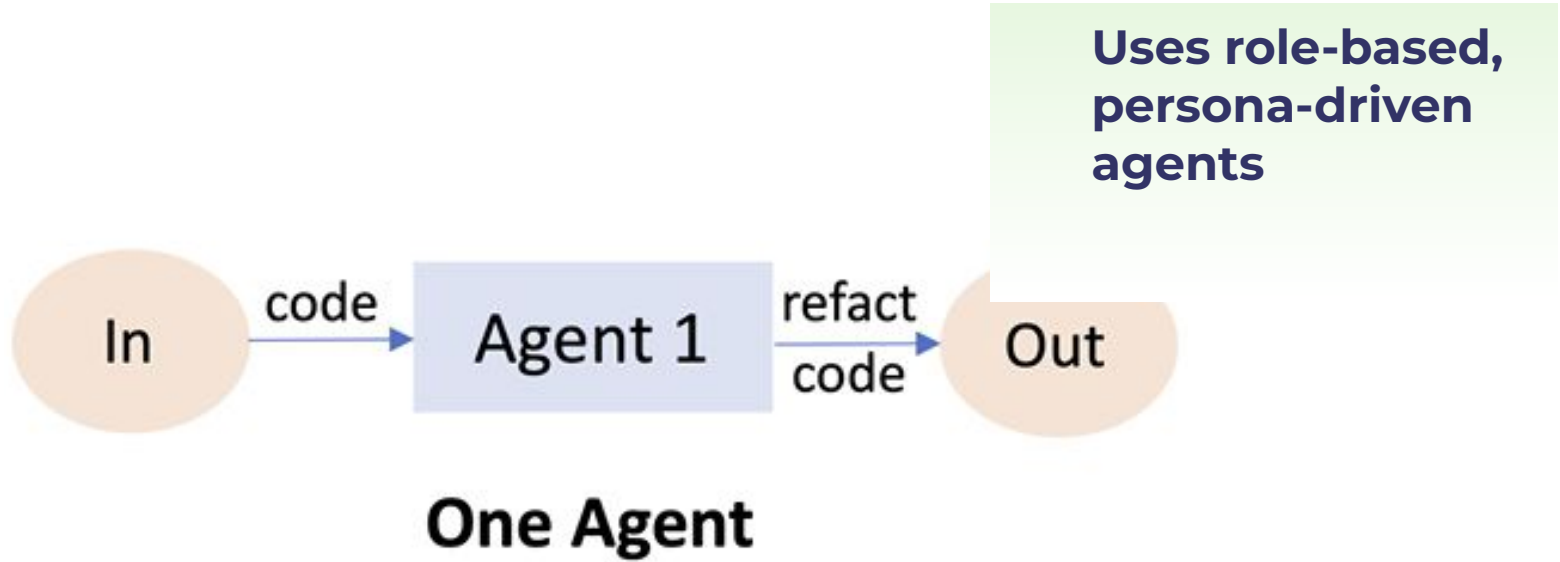


Rule-Based Approach

# TemPy Framework Workflow

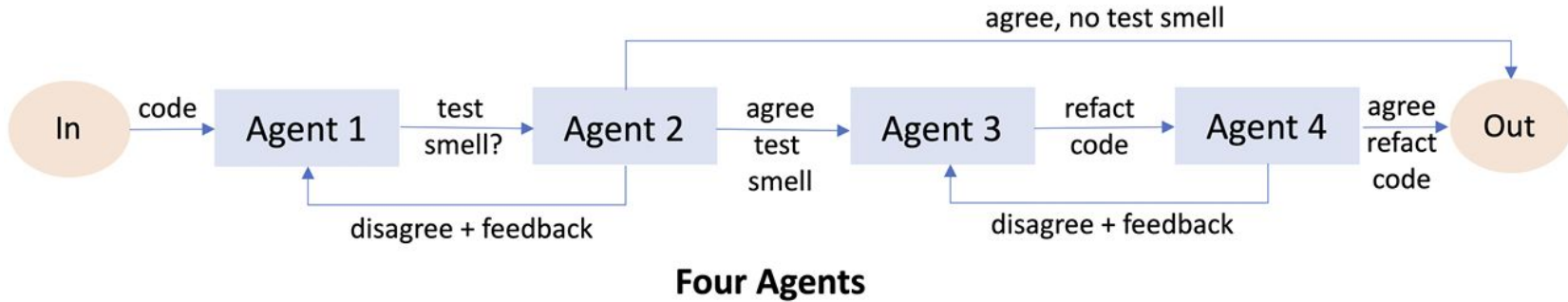
1. **Parsing** : Converts each test file into an **Abstract Syntax Tree (AST)** & represents the code's syntax as a tree structure.
2. **Traversal**: Visits each node in the AST (functions, statements, method calls).
3. **Pattern Matching**: Applies **rules for known test smells**: e.g.
  - `time.sleep()` → **Sleepy Test**
  - Test methods without asserts → **Undefined Test**
4. **Data Recording**: smell type, method name, line number
5. **Report Generation**

# Test smell detection & Refactoring



Single Agent Workflow (LLM Based Approach)

# Test smell detection & Refactoring (cont.)



Multi Agent Workflow (LLM Based Approach))

# Persona-Driven Agent (Example)

You are a coding assistant with many years of experience that detects test smells.

`test_smell_definition_and_refactoring`

Your goal is to determine if the provided test code exhibits the test smell "`test_smell_name`".

`code`

Next I may give you further details.

`agent_feedback`

If the test code contains `test_smell_name`, respond with EXACTLY "YES" on the first line and explain why. Ignore code comments. If it does not contain, say EXACTLY "NO" on the first line and explain why not.

# Persona-Driven Agent (Example)

You are a coding expert reviewing the detection of a test smell.  
Consider the following test smell.

`test_smell_definition_and_refactoring`

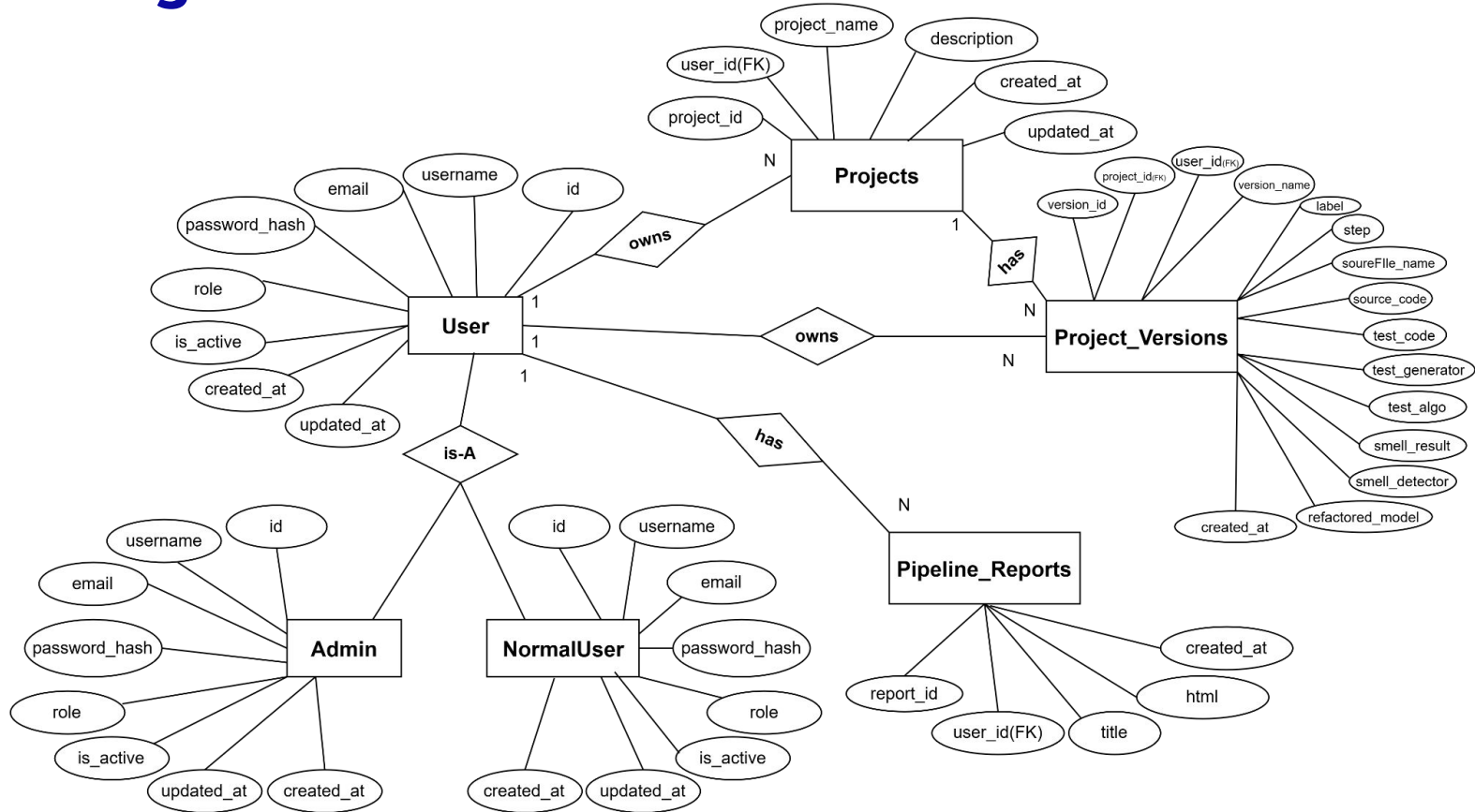
A previous agent analyzed the following test code.  
code

It gave the following answer.

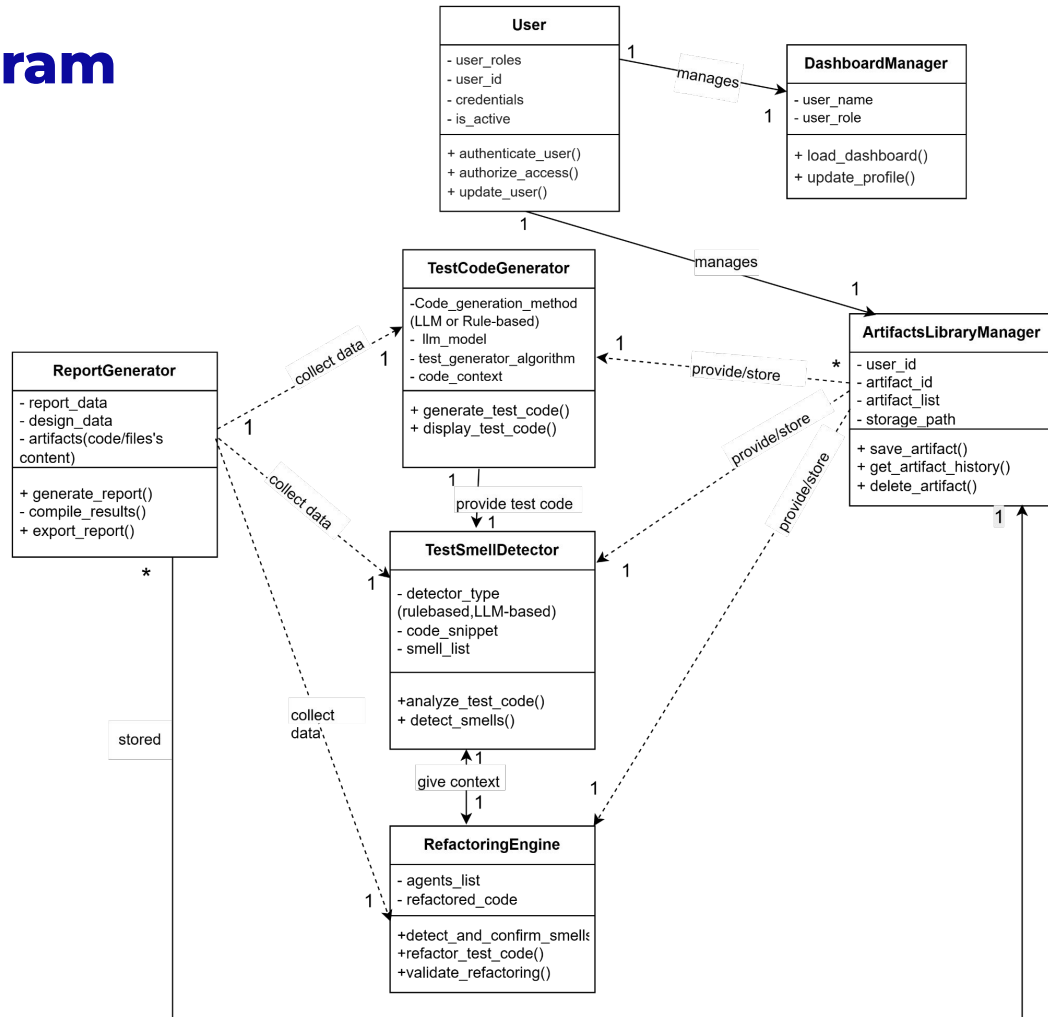
`agent_answer`

Your goal is to evaluate if the previous detection by another agent is correct and justified. Ignore code comments. If you do not agree, answer NO and explain what's wrong with it and what to correct. If yes, just say YES.

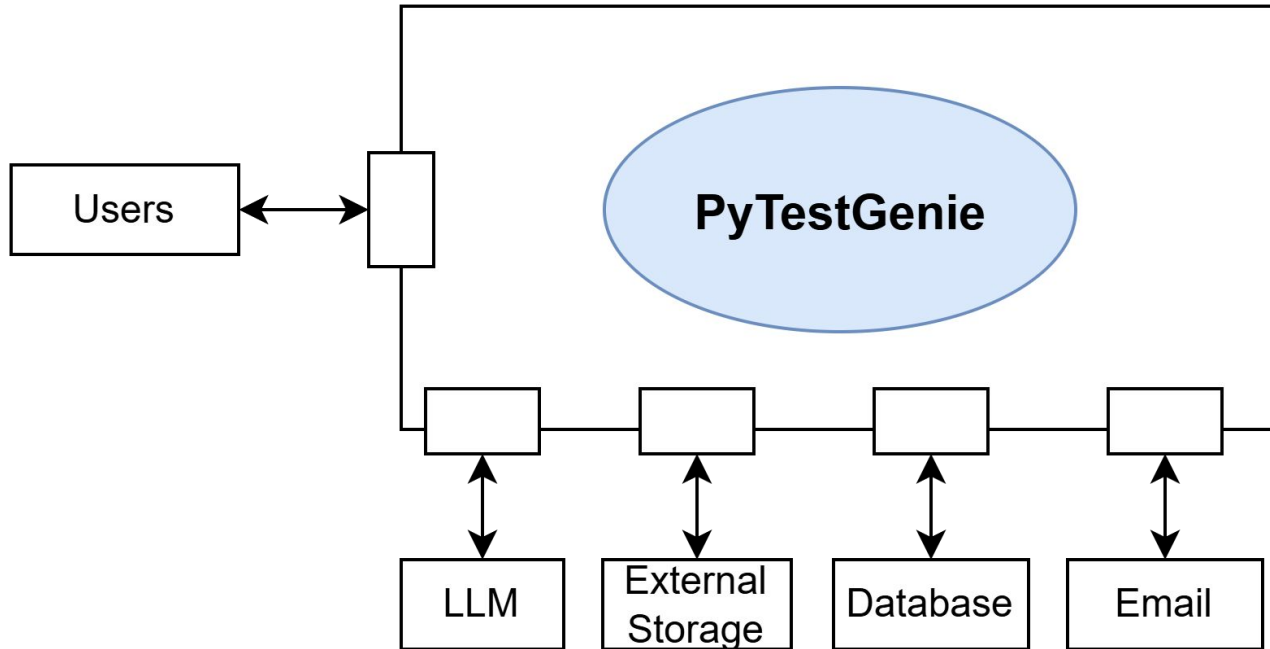
# ER Diagram



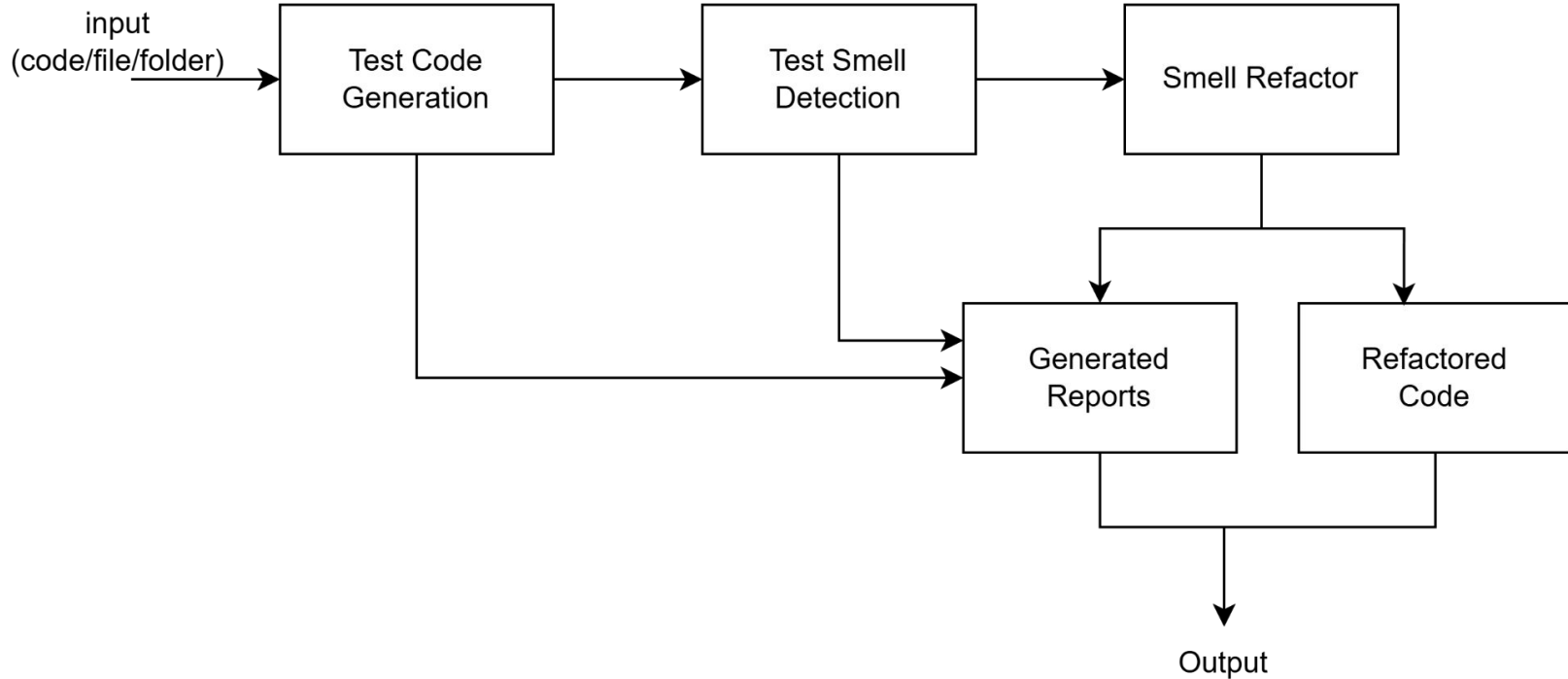
# CRC Diagram



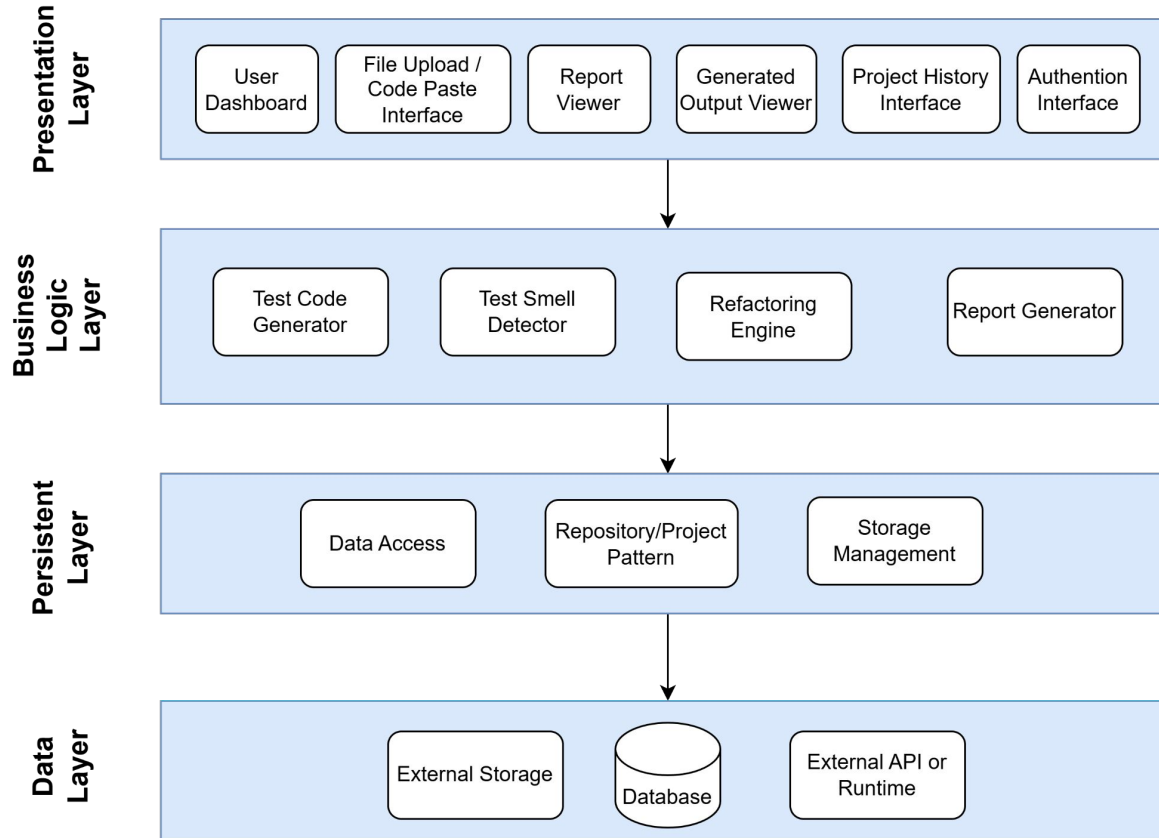
# Architectural Context Diagram



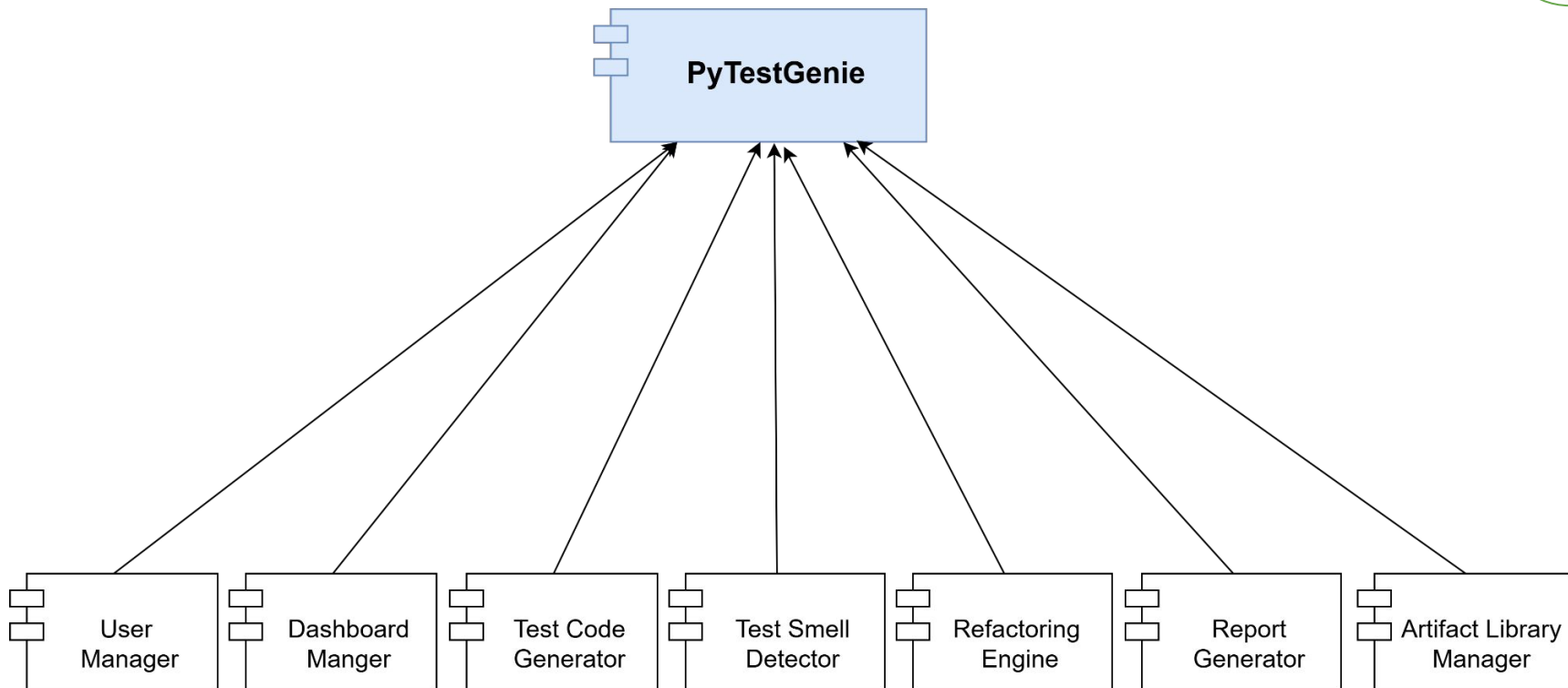
# Pipe and Filter Architecture



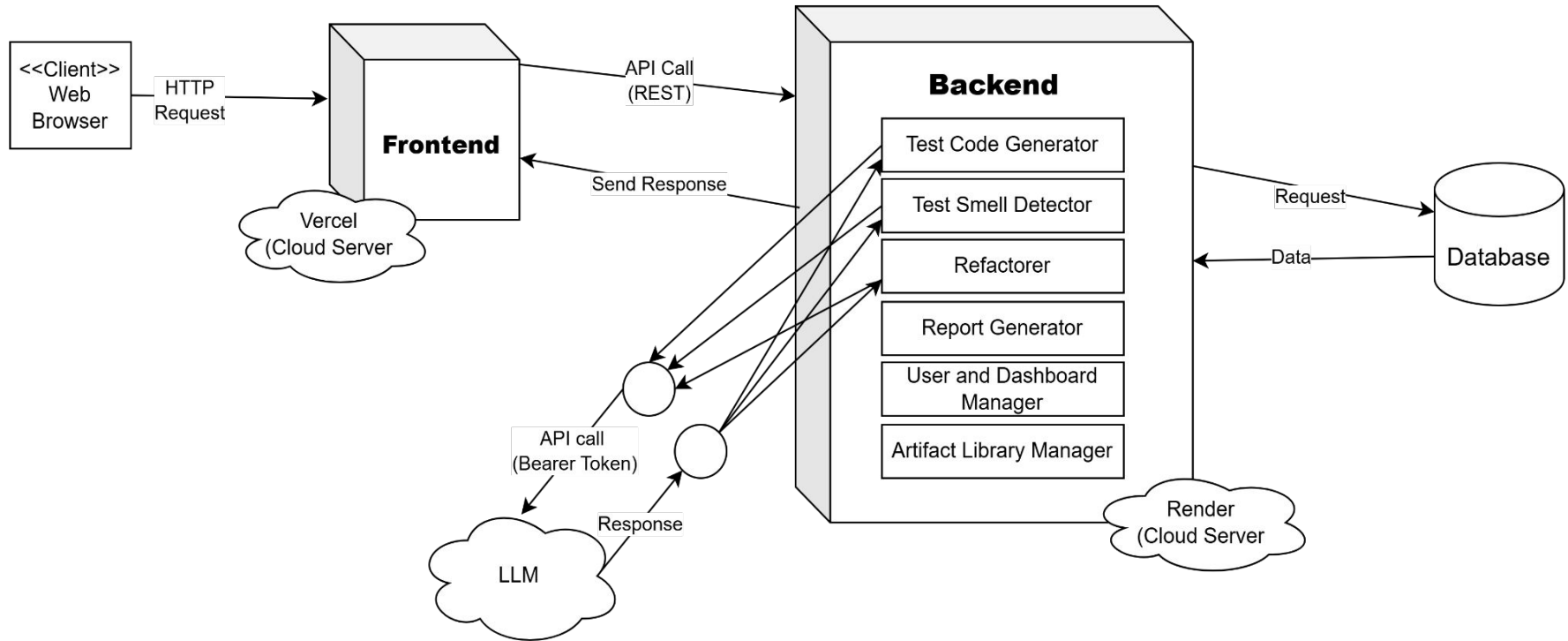
# Layered Architecture



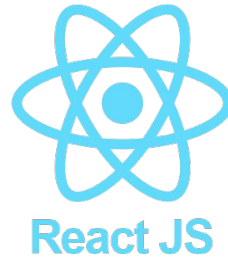
# Top level Components Basic Architecture



# Deployment Diagram



# Tools & Technologies



# Let's see what the reports say



- Implementation Details
- User Interfaces
- User Manual
- Test reports ([Test Report](#) )

**Public URL of the web tool:** <https://py-test-genie-v1.vercel.app/>

**Github Repository link:**

<https://github.com/AmitRoy01/PyTestGenie>

# Timeline

| TASKS                    | OCTOBER |     |     |     | NOVEMBER |     |     |     | DECEMBER |     |     |     | JANUARY |     |     |     | FEBRUARY |     |     |     |
|--------------------------|---------|-----|-----|-----|----------|-----|-----|-----|----------|-----|-----|-----|---------|-----|-----|-----|----------|-----|-----|-----|
|                          | 1st     | 2nd | 3rd | 4th | 1st      | 2nd | 3rd | 4th | 1st      | 2nd | 3rd | 4th | 1st     | 2nd | 3rd | 4th | 1st      | 2nd | 3rd | 4th |
| Prerequisite knowledge   |         |     |     |     |          |     |     |     |          |     |     |     |         |     |     |     |          |     |     |     |
| Requirment Specification |         |     |     |     |          |     |     |     |          |     |     |     |         |     |     |     |          |     |     |     |
| Project Design           |         |     |     |     |          |     |     |     |          |     |     |     |         |     |     |     |          |     |     |     |
| Development              |         |     |     |     |          |     |     |     |          |     |     |     |         |     |     |     |          |     |     |     |
| Testing                  |         |     |     |     |          |     |     |     |          |     |     |     |         |     |     |     |          |     |     |     |
| Final Report Writing     |         |     |     |     |          |     |     |     |          |     |     |     |         |     |     |     |          |     |     |     |

# References

- [1] S. Lukasczyk and G. Fraser, “Pynguin: Automated unit test generation for python,” in **Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings, ser. ICSE ’22, Pittsburgh, Pennsylvania: Association for Computing Machinery, 2022, pp. 168–172, isbn: 9781450392235. doi: 10.1145/3510454.3516829. [Online]. Available: <https://doi.org/10.1145/3510454.3516829>**
- [2] R Melo, P Simões, R Gheyi, M d'Amorim, M Ribeiro, G Soares, E Almeida, E Soares, “Agentic LMs: Hunting Down Test Smells”arXiv preprint arXiv:2504.07277, 2025.
- [3] D Fernandes, I Machado, R Maciel, “TEMPY: Test smell detector for Python”. **Proceedings of the XXXVI Brazilian Symposium on Software Engineering, 2022**
- [4] **Automated Test Case Generation as a Many-Objective Optimisation Problem with Dynamic Selection of the Targets**
- [5] **Many Independent Objective (MIO) Algorithm for Test Suite Generation**
- [6]PyNose: A Test Smell Detector For Python, Tongjie Wang, Yaroslav Golubev, Oleg Smirnov, Jiawei Li, Timofey Bryksin, Iftekhar Ahmed.



# Thanks!

Please let me know if you have any questions...